

Homework 05

zhijing huang

2026-09-27

Question 1

a. $f(x_0)$ is 0.5942177.

```
library(FNN)
set.seed(43251)
n = 400
p = 3
k_val = seq(1, 29, 2)
x0 <- c(0.5, 0.7, 1)
f0 <- 0.5*x0[1] + sin(x0[2]) - 0.3*x0[3]^2
f0
## [1] 0.5942177
X = matrix(rnorm(n*p), nrow = n, ncol = p)
y = 0.5*X[, 1] + sin(X[, 2]) - 0.3*X[, 3]^2 + rnorm(n, sd = 0.1)
pred = numeric(length(k_val))
for (m in seq_along(k_val)) {
  k = k_val[m]
  knn.fit = knn.reg(train = X, test = x0, y = y,
                    k = k, algorithm = "brute")
  pred[m] = knn.fit$pred
}
data.frame(k = k_val, prediction = pred, truth = f0)
##      k prediction      truth
## 1    1  0.7888302 0.5942177
## 2    3  0.6621674 0.5942177
## 3    5  0.6642853 0.5942177
## 4    7  0.6100117 0.5942177
## 5    9  0.6318671 0.5942177
## 6   11  0.4925114 0.5942177
## 7   13  0.5222989 0.5942177
## 8   15  0.5810203 0.5942177
## 9   17  0.5738171 0.5942177
## 10  19  0.5751479 0.5942177
## 11  21  0.5800517 0.5942177
## 12  23  0.5519177 0.5942177
## 13  25  0.5214556 0.5942177
## 14  27  0.4952957 0.5942177
## 15  29  0.4972667 0.5942177
```

b. Among the prediction matrix, each column contains 200 estimates for the same target. $-f(x_0)$ measures the bias, and its corresponding variance is the sampling variance. The average squared difference of $f(x_0)$ is the MSE.

```
nsim = 200
predict_matrix <- matrix(NA, nrow = nsim, ncol = length(k_val))
for (i in 1:nsim) {
  X <- matrix(rnorm(n * p), nrow = n, ncol = p)

  y <- 0.5 * X[, 1] + sin(X[, 2]) - 0.3 * X[, 3]^2 +
    rnorm(n, sd = 0.1)

  predict_matrix[i, ] <- sapply(k_val, function(k) {
    fit <- FNN::knn.reg(
      train = X,
      test = x0,
      y = y,
      k = k,
      algorithm = "brute"
    )
    fit$pred
  })
}
```

```

colnames(predict_matrix) <- paste0("k_", k_val)
mean_prediction <- colMeans(predict_matrix)
bias <- mean_prediction - f0
variance <- colMeans(
  sweep(predict_matrix, 2, mean_prediction, "-")^2
)
mse <- colMeans((predict_matrix - f0)^2)
data.frame(k = k_val, bias = bias, variance = variance, MSE = mse)
##      k      bias      variance      MSE
## k_1  1 -0.001248923  0.037181204  0.03718276
## k_3  3 -0.029361680  0.019831580  0.02069369
## k_5  5 -0.040815728  0.014731018  0.01639694
## k_7  7 -0.045589732  0.010865566  0.01294399
## k_9  9 -0.051867114  0.009467440  0.01215764
## k_11 11 -0.062360901  0.009506180  0.01339506
## k_13 13 -0.067797856  0.009397702  0.01399425
## k_15 15 -0.070662763  0.008542311  0.01353554
## k_17 17 -0.081499864  0.008213727  0.01485595
## k_19 19 -0.089385641  0.008429774  0.01641957
## k_21 21 -0.096080776  0.008413451  0.01764497
## k_23 23 -0.102803643  0.008103017  0.01867161
## k_25 25 -0.107847900  0.008091147  0.01972232
## k_27 27 -0.114428351  0.007777947  0.02087180
## k_29 29 -0.121688194  0.007474576  0.02228259
max(abs(mse - bias^2 - variance))
## [1] 6.765422e-17

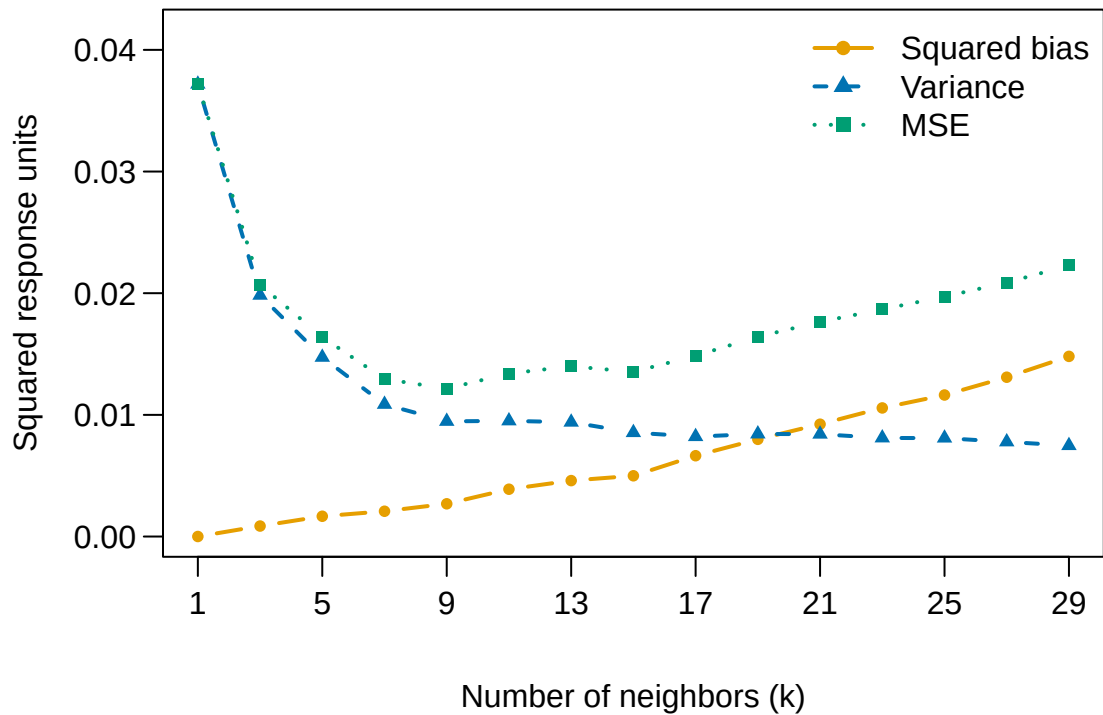
```

```

local({
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))
  par(mai = c(1.0, 1.1, 0.65, 0.5), ps = 12,
      cex.axis = 1, cex.lab = 1, cex.main = 14 / 12,
      mgp = c(3.1, 0.7, 0), las = 1)
  colors <- c("#E69F00", "#0072B2", "#009E73")
  matplot(
    k_val, cbind(bias^2, variance, mse),
    type = "b", pch = c(16, 17, 15), lty = c(1, 2, 3),
    lwd = 2, cex = 0.8, col = colors, xaxt = "n",
    xlab = "Number of neighbors (k)",
    ylab = "Squared response units",
    main = "KNN Bias-Variance Trade-off",
    ylim = c(0, 1.12 * max(bias^2, variance, mse))
  )
  axis(1, at = k_val[seq(1, length(k_val), by = 2)])
  legend(
    "topright", legend = c("Squared bias", "Variance", "MSE"),
    col = colors, pch = c(16, 17, 15), lty = c(1, 2, 3),
    lwd = 2, bty = "n", cex = 1
  )
})

```

KNN Bias–Variance Trade-off



Question 2

a.

```
library(glmnet)
set.seed(43252)
n <- 400
p <- 30
lambda <- 10^seq(0, -4, length.out = 41)
results <- data.frame(
  Setting = c("Independent", "Correlated"),
  KNN_MSE = NA_real_,
  Lasso_MSE = NA_real_,
  Lambda = NA_real_,
  Nonzero = NA_integer_
)
simulation <- function(l) {
  Sigma <- if (l == 1) diag(p) else 0.8 + 0.2 * diag(p)

  X <- matrix(rnorm((n + 1000) * p), ncol = p) %%% chol(Sigma)
  y <- 0.5 * X[, 1] + sin(X[, 2]) - 0.3 * X[, 3]^2 +
    rnorm(n + 1000, sd = 0.1)

  test.X <- X[(n + 1):(n + 1000), ]
  test.y <- y[(n + 1):(n + 1000)]
  X <- X[1:n, ]
  y <- y[1:n]

  knn.fit <- knn.reg(X, test.X, y, k = 5, algorithm = "brute")
  standardize <- function(train, newdata) {
    means <- colMeans(train)
    centered <- sweep(train, 2, means, "-")
    scales <- sqrt(colMeans(centered^2))

    list(
      train = sweep(centered, 2, scales, "/"),
      newdata = sweep(
        sweep(newdata, 2, means, "-"),
        2, scales, "/"
      )
    )
  }
}
```

```

}
fold <- sample(rep(1:10, each = n / 10))
cv.mse <- matrix(NA_real_, nrow = 10, ncol = length(lambda))
for (m in 1:10) {
  scaled <- standardize(
    train = X[fold != m, ],
    newdata = X[fold == m, ]
  )
  X.fit <- scaled$train
  X.valid <- scaled$newdata
  y.mean <- mean(y[fold != m])

  lasso.fit <- glmnet(
    X.fit, y[fold != m] - y.mean, alpha = 1, lambda = lambda,
    intercept = FALSE, standardize = FALSE
  )
  y.hat <- predict(lasso.fit, newx = X.valid, s = lambda) + y.mean
  cv.mse[m, ] <- colMeans((y[fold == m] - y.hat)^2)
}
lambda.min <- lambda[which.min(colMeans(cv.mse))]

scaled <- standardize(train = X, newdata = test.X)
X.fit <- scaled$train
test.X.fit <- scaled$newdata
y.mean <- mean(y)
lasso.fit <- glmnet(
  X.fit, y - y.mean, alpha = 1, lambda = lambda,
  intercept = FALSE, standardize = FALSE
)
test.pred <- as.vector(
  predict(lasso.fit, newx = test.X.fit, s = lambda.min)
) + y.mean
c(
  KNN_MSE = mean((test.y - knn.fit$pred)^2),
  Lasso_MSE = mean((test.y - test.pred)^2),
  Lambda = lambda.min,
  Nonzero = sum(coef(lasso.fit, s = lambda.min)[-1, 1] != 0)
)
}
results[, c("KNN_MSE", "Lasso_MSE", "Lambda", "Nonzero")] <-
do.call(rbind, lapply(1:2, simulation))
results
##      Setting   KNN_MSE Lasso_MSE      Lambda Nonzero
## 1 Independent 0.5564800 0.2601745 0.02511886      11
## 2 Correlated 0.1608514 0.2259450 0.01000000      16

```

- b. Lasso has the smallest test MSE with independent covariates, where many coordinates that don't affect the response which is a contribution to the Euclidean distance, and KNN has the smallest test MSE with correlated covariates. In the independent setting, the 27 predictors that do not affect the response still contribute to Euclidean distance, making KNN's neighborhoods less useful. In the correlated setting, these predictors share information with the relevant predictors, which can help KNN find informative neighbors. However, correlation does not always improve KNN's performance.

Unlike KNN, which uses all supplied covariates to calculate distance, lasso learns a coefficient for each predictor and shrinks some coefficients to zero. It can therefore reduce the influence of unhelpful predictors.

c. Because the true relationship is not linear, the fitted lasso regression will result in deviations on predictors. The lasso intercept can only account for the average, not the variation. On the opposite, KNN does not impose a linear form, instead, it focuses on whether it can find informative neighbours. Thus, the comparison reflects both model flexibility and how the methods handle predictors. These results do not show that correlation always helps KNN or that either method is always better, because performance depends on the data structure and this experiment uses only one dataset per setting.

Question 3

a.

```

set.seed(43254)
p = 100
m_val <- c(3,30)
k_val <- seq(2, 82, 8)
mse_3a <- matrix(
  NA_real_,

```

```

nrow = length(k_val),
ncol = length(m_val),
dimnames = list(NULL, c("m_3", "m_30"))
)
for (m in m_val) {
  A <- matrix(runif(p*m, -2, 2), nrow = p, ncol = m)
  Z <- matrix(rnorm(400*m), nrow = 400, ncol = m)
  X = Z %*% t(A) + matrix(rnorm(400*p), nrow = 400, ncol = p)
  y = Z[, 1] + Z[, 2] + rnorm(400)
  for (k in k_val) {
knn.fit = knn.reg(train = X[1:200, ], test = X[201:400, ], y = y[1:200], k = k, algorithm =
  "brute")
    mse_3a[match(k, k_val), match(m, m_val)] = mean((y[201:400] - knn.fit$pred)^2)
  }
}
data.frame(k = k_val, m3 = mse_3a[, 1], m30 = mse_3a[, 2])
##      k      m3      m30
## 1  2  1.449265  3.457456
## 2 10  1.253345  2.583372
## 3 18  1.292923  2.533071
## 4 26  1.379955  2.524038
## 5 34  1.448416  2.512187
## 6 42  1.505892  2.560783
## 7 50  1.590057  2.562911
## 8 58  1.660186  2.593970
## 9 66  1.739597  2.607162
## 10 74  1.816963  2.651181
## 11 82  1.871802  2.672724

```

- b. $m = 3$ has a smaller average test MSE than $m = 30$. Large neighbours will blur the regression into a flatter shape, while small neighbourhood is less local in latent variables. We will not be able to choose which model is better, as they are simulation averages over independent experiments.

```

nsim = 50
test_mse <- array(NA, dim = c(nsim, length(k_val), 2))
for (l in 1:nsim) {
  for (m in m_val) {
    A <- matrix(runif(p*m, -2, 2), nrow = p, ncol = m)
    Z <- matrix(rnorm(400*m), nrow = 400, ncol = m)
    X = Z %*% t(A) + matrix(rnorm(400*p), nrow = 400, ncol = p)
    y = Z[, 1] + Z[, 2] + rnorm(400)
    for(k in k_val){
knn.fit <- FNN::knn.reg(train = X[1:200, ], test = X[201:400, ], y = y[1:200], k = k, algorithm =
  "brute")
      predictions <- knn.fit$pred
      mse <- mean((y[201:400] - predictions)^2)
      k_position <- match(k, k_val)
      m_position <- match(m, m_val)

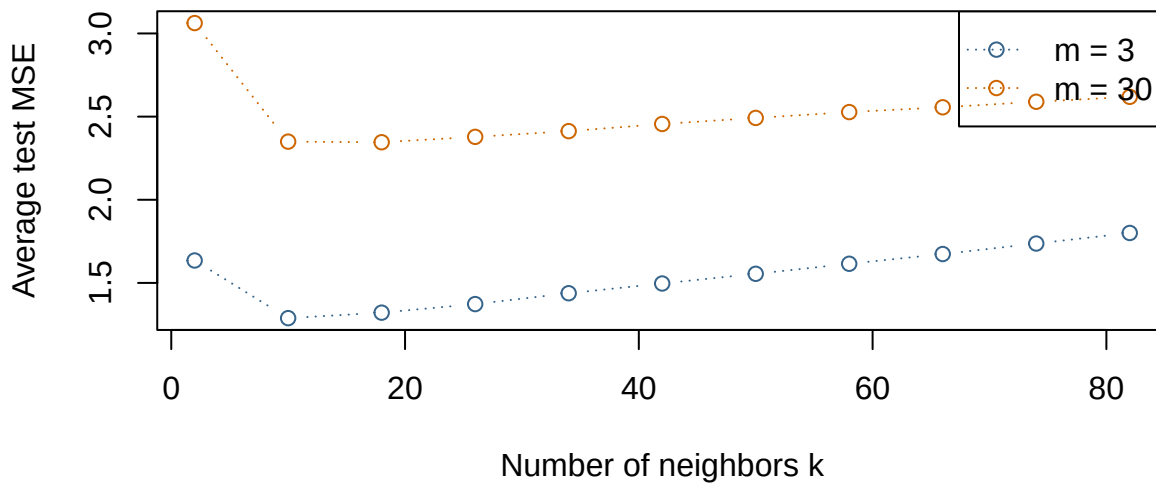
      test_mse[l, k_position, m_position] <- mse
    }
  }
}
mean_mse = apply(test_mse, c(2, 3), mean)
data.frame(k = k_val, m3 = mean_mse[, 1], m30 = mean_mse[, 2])
##      k      m3      m30
## 1  2  1.634822  3.062662
## 2 10  1.288062  2.349524
## 3 18  1.321139  2.345840
## 4 26  1.372628  2.378219
## 5 34  1.437850  2.412709
## 6 42  1.496497  2.455664
## 7 50  1.554715  2.492101
## 8 58  1.615062  2.527212
## 9 66  1.673644  2.556039
## 10 74  1.736733  2.589309
## 11 82  1.800274  2.618853

```

```

matplot(k_val, mean_mse, type = "b", pch = c(1, 1), lty = 3,
  col = c("steelblue4", "darkorange3"),
  xlab = "Number of neighbors k", ylab = "Average test MSE")
legend("topright", c("m = 3", "m = 30"),
  col = c("steelblue4", "darkorange3"), lty = 3, pch = c(1, 1))

```



- c. Both settings have 100 observed predictors, but their shared structure comes from different numbers of latent variables, so 100 measurements can contain a relatively simple or much more complicated signal. In this case, when ($m=3$), there are fewer latent directions unrelated to the response than when ($m=30$). These additional directions can make observations with similar values of (Z_1) and (Z_2) appear far apart, making Euclidean neighborhoods less informative for predicting (Y).

The measurement noise perturbs every observed predictor and can change the nearest neighbors. Without this noise, AZ_i lies in a linear space of dimension at most (m). With independent noise in all 100 coordinates, the observations are no longer confined to that space. Thus, the data contain a lower-dimensional signal surrounded by noise.

Question 4

- a. classifier

```
library(ElemStatLearn)
data(zip.train)
data(zip.test)
#test
dim(zip.train) # 7291 257
## [1] 7291 257
dim(zip.test) # 2007 257
## [1] 2007 257
euclidean_distance <- function(x0, X) {
  sqrt(rowSums(sweep(X, 2, x0, "-")^2))
}
manhattan_distance <- function(x0, X) {
  rowSums(abs(sweep(X, 2, x0, "-")))
}
x <- zip.train[1:1000, -1]
y <- zip.train[1:1000, 1]
x_test <- zip.test[1:500, -1]
y_test <- zip.test[1:500, 1]
knn_clas <- function(x, y, x_test, distance) {
  y_hat <- apply(x_test, 1, function(x0) {
    d <- distance(x0, x)
    neighbors <- order(d, seq_len(nrow(x)))[1:5]

    votes <- tabulate(y[neighbors] + 1, nbins = 10)
    which.max(votes) - 1
  })
  y_hat
}
y_hat <- cbind(Euclidean = knn_clas(x, y, x_test, euclidean_distance),
               Manhattan = knn_clas(x, y, x_test, manhattan_distance))
```

- b. For Euclidean distance, the test error is 0.122, and for Manhattan distance, the test error is 0.138. The most often confused digit is 2 predicted as 0, which is seven errors under the Euclidean and six errors under the Manhattan. Other frequent errors include 9 as 7, and 4 as 9.

```
for (l in 1:2) {
  cat(colnames(y_hat)[l], "test error:", mean(y_hat[,l] != y_test), "\n")
  predicted <- factor(y_hat[, l], levels = 0:9)
  actual <- factor(y_test, levels = 0:9)

  confusion_matrix <- table(
    Predicted = predicted,
    True = actual
  )

  print(confusion_matrix)
}

## Euclidean test error: 0.122
##      True
## Predicted  0  1  2  3  4  5  6  7  8  9
##      0 118  0  7  1  0  4  2  0  2  0
##      1  0 66  0  0  2  0  0  0  0  1
##      2  0  0 39  1  0  0  2  0  0  1
##      3  0  0  0 23  0  4  0  0  2  0
##      4  0  0  1  0 27  1  1  1  0  1
##      5  0  0  0  1  0 13  0  0  0  0
##      6  0  2  0  0  0  1 41  0  2  0
##      7  0  0  0  0  1  1  0 34  0  4
##      8  0  0  3  2  0  2  0  0 37  0
##      9  1  0  0  1  4  0  0  1  1 41

## Manhattan test error: 0.138
##      True
## Predicted  0  1  2  3  4  5  6  7  8  9
##      0 118  0  6  1  0  5  3  0  4  0
##      1  0 67  2  0  2  0  1  0  0  1
##      2  0  0 35  0  0  0  2  0  0  0
##      3  0  0  1 24  0  4  0  0  2  0
##      4  0  0  1  1 27  1  0  1  0  1
##      5  0  0  0  1  0 11  0  0  0  0
##      6  0  1  0  0  0  2 40  0  2  0
##      7  0  0  2  1  1  0  0 34  0  5
##      8  0  0  2  0  0  3  0  0 34  0
##      9  1  0  1  1  4  0  0  1  2 41
```

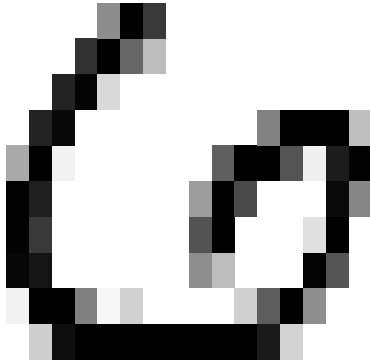
- c. The three displayed images have true labels 6, 2, and 4, but Euclidean 5NN predicts 0, 0, and 7. These mistakes are understandable because the shapes are not standard enough, as 6 looked more like a 0, 2 cannot find its flat stroke, and 4 shaped like a Y.

The Euclidean distance and Manhattan distance disagrees on 22 images, where Euclidean distance is correct and Manhattan distance is wrong on 11 images; and Manhattan distance is correct and Euclidean distance is wrong on 3 images. On the 8 they were both wrong they predicted different digits, thus, they can have similar total error while making mistakes on different images.

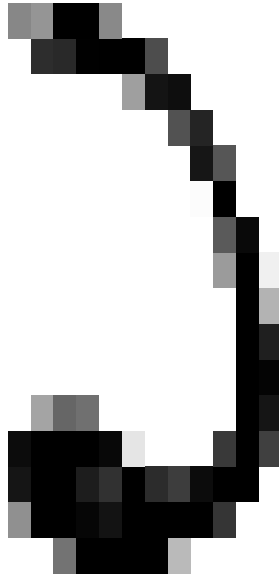
```
correct_euclidean <- y_hat[, 1] == y_test
correct_manhattan <- y_hat[, 2] == y_test
incorrect <- which(!correct_euclidean)[1:3]
old_par <- par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))
for (i in incorrect) {
  image(
    zip2image(zip.test, i),
    col = gray(256:0 / 256),
    xlim = c(0, 1),
    xlab = "",
    ylab = "",
    axes = FALSE,
    main = sprintf("True: %s\nPredicted: %s", y_test[i], y_hat[i, 1])
  )
}

## [1] "digit 6 taken"
## [1] "digit 2 taken"
## [1] "digit 4 taken"
```

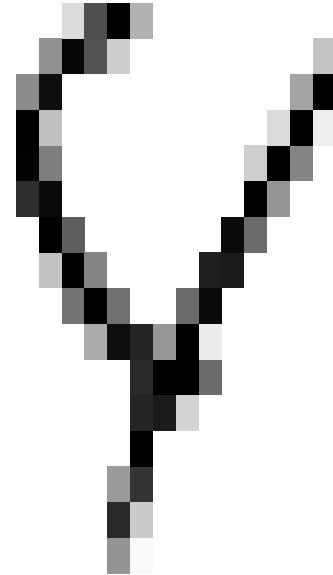
True: 6
Predicted: 0



True: 2
Predicted: 0



True: 4
Predicted: 7



```
par(old_par)
cat("Different predictions:", sum(y_hat[, 1] != y_hat[, 2]), "\n")
## Different predictions: 22
table(
  Euclidean_correct = correct_euclidean,
  Manhattan_correct = correct_manhattan
)
##               Manhattan_correct
## Euclidean_correct FALSE TRUE
##          FALSE      58      3
##          TRUE       11     428
```

Question 5

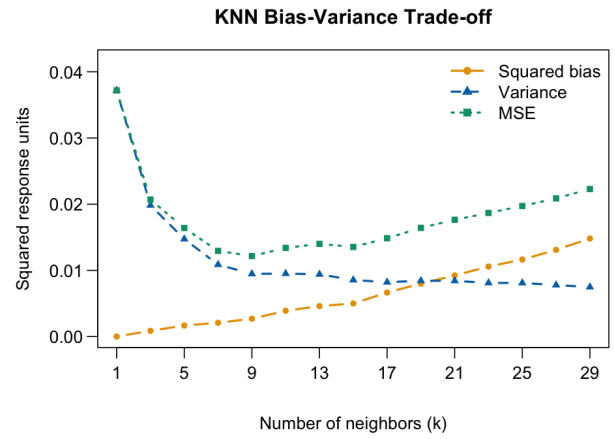
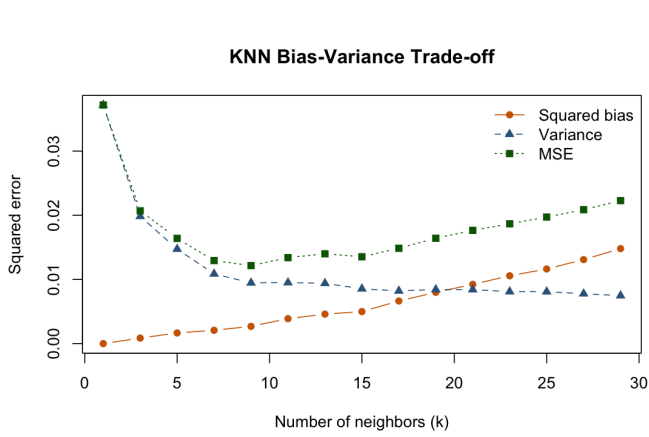
```
---
name: format-homework-plots
description: Use only when the user explicitly calls this skill to improve the formatting of a
  STAT 432 homework plot.
---

# STAT 432 Plot Formatting

- Preserve the plotted data, calculations, and statistical meaning.
- Leave a 0.5 inch margin for titles, axis labels, tick labels, and legends without clipping.
- Label axes clearly, include units when relevant, and use readable tick spacing and labels.
- Use a colorblind-friendly palette and distinguish curves with line styles when helpful.
- Start with a 6.5 by 4.5 inch figure, 12-point labels, and a 14-point title; adjust for
  multiple panels.
- Apply these rules to the existing R plotting code.
```

Part b: Apply and compare the skill

I explicitly requested the `format-homework-plots` skill for the Question 1 plot. The skill was installed in my personal skills folder. The revised plot uses the same simulated values, with a colorblind-friendly palette, thicker lines, distinct line styles and markers, 12-point labels, a 14-point title, and margins of at least 0.5 inch.



The original plot is on the left and the revised plot is on the right. The revision changes presentation only; the estimates and statistical conclusions are unchanged.

Personal assessment: I think the revised plot is more suitable for most cases, though the original one looks fine as well. Codex said it added “colorblind-friendly colors, thicker lines, explicit text sizes, and more label space”. Personally I felt the colors went brighter, so it may be better to those who wants to take a quick look at the data. The x axis(number of neighbours) are also labeled more precisely, which is a progress. The y-axis now has forward-facing verbal, which is more applicable.